Isolation Levels

snapshot isolation, serializable, and write skew

Watch the 4-min video

In one line: isolation levels decide which concurrency anomalies you can see. Read committed is the common default; snapshot isolation (repeatable read in Postgres and MySQL) stops most anomalies but still allows **write skew**, which only serializable or explicit locks prevent.

1 The doctors problem

Two doctors are on call tonight. The rule: **at least one must stay on call**. Both feel unwell, and at the same moment each one checks: "is someone else on call?" Yes, one other. So each takes themselves off. Both transactions succeed, and now nobody is on call.

Every single row was updated correctly. Nothing crashed. Yet the rule is broken. This is **write skew**, and whether it can happen depends on your **isolation level**.

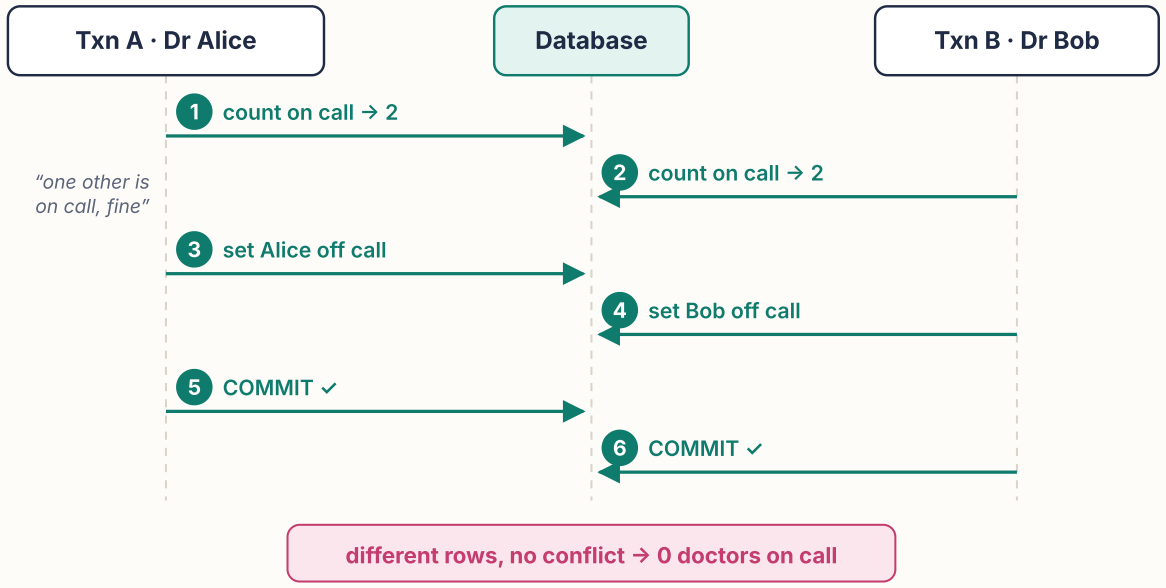


Figure 1. Each transaction reads the same data, decides "safe", and writes a different row. No row is written twice, so snapshot isolation sees no conflict.

2 The anomalies

Anomaly	What happens
Dirty read	You see another transaction's **uncommitted** change, which might still be rolled back.
Non-repeatable read	You read a row twice and get different values, because someone committed in between.
Phantom	You run the same query twice and **new rows** appear (or disappear).
Lost update	Two transactions read, modify and write the same value; one silently overwrites the other.
Write skew	Two transactions read the same data, then update **different** rows, breaking a rule that spans both.

3 The four standard levels, and what databases really do

- **Read uncommitted** allows dirty reads. Rarely used (Postgres treats it as read committed).
- **Read committed** prevents dirty reads. It is the default in **Postgres, Oracle and SQL Server**, but allows non-repeatable reads and phantoms.
- **Repeatable read** keeps the rows you have read stable. In the SQL standard, phantoms can still appear.
- **Serializable** promises a result equal to *some* one-at-a-time order: no anomalies at all.

In practice, most databases implement repeatable read as **snapshot isolation**: each transaction reads from a consistent snapshot taken when it started. Repeatable read is **MySQL's default**. In Postgres, repeatable read is snapshot isolation too: it also hides phantoms, and if two transactions update the **same row**, it aborts one of them, which prevents lost updates. But it still allows write skew, because the two transactions update **different** rows.

	Dirty read	Non-repeat.	Phantom	Lost update	Write skew
Read uncommitted	possible	possible	possible	possible	possible
Read committed	prevented	possible	possible	possible	possible
Repeatable read (std)	prevented	prevented	possible	—	—
Snapshot (Postgres RR)	prevented	prevented	prevented	prevented	possible
Serializable	prevented	prevented	prevented	prevented	prevented

— the SQL standard does not define these. MySQL's repeatable read is snapshot-based but can still lose a read-modify-write update.

Figure 2. Which anomaly each level allows. The row to remember: snapshot isolation prevents everything except write skew.

4 Preventing write skew

Back to the doctors under snapshot isolation. Both transactions take a snapshot. Each counts two doctors on call, and each updates only its own row. No row is written twice, so there is no conflict to detect, and both commit. Three fixes:

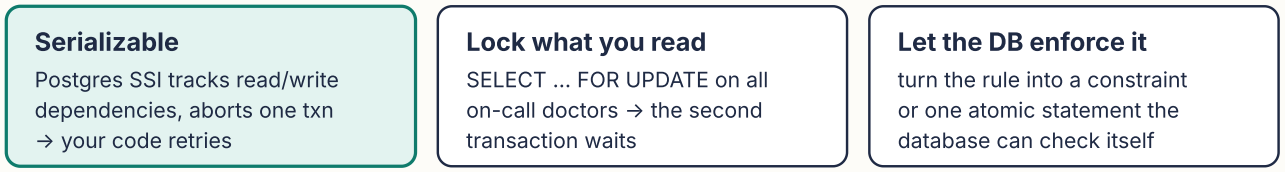


Figure 3. Three ways to stop write skew. Serializable is the most general; locks and constraints are targeted.

WORKED EXAMPLE: THE DOCTORS, DONE SAFELY

Option 1, serializable. Run the transaction at `SERIALIZABLE`. Postgres implements it as **serializable snapshot isolation** (SSI): when the second doctor commits, it fails with a serialization failure (SQLSTATE `40001`). The retry re-reads, sees only one doctor on call, and refuses.

Option 2, lock the rows you read. The second transaction blocks on the first one's locks, then re-checks after it commits:

```
BEGIN;
SELECT id FROM doctors
 WHERE shift_id = 7 AND on_call
 FOR UPDATE;              -- locks both rows
-- app: if 2 or more rows, continue; else abort
UPDATE doctors SET on_call = false WHERE id = :me;
COMMIT;
```

Note: Postgres does not allow `FOR UPDATE` with `count(*)`, so select the rows and count them in code.

5 Choosing a level

Situation	Use
Most simple reads and writes	**Read committed** (the usual default)
Counters, read-modify-write	Atomic updates (`SET n = n + 1`) or row locks
A consistent report across several queries	**Repeatable read** (one snapshot for all of them)
Invariants that span multiple rows	**Serializable**, with retry logic around every transaction

COMMON MISTAKES

- Assuming “repeatable read” means the same thing everywhere. Check your database's actual behaviour.
- Turning on serializable without a retry loop: serialization failures are normal, not bugs.
- Believing every row update was correct, so the data must be. Write skew breaks rules between rows.
- Using `FOR UPDATE` to guard against rows that don't exist yet. It locks existing rows only; inserts can still slip in.

6 Key terms

Isolation level: how much concurrent work a transaction can see

Snapshot: a consistent view as of one moment

Snapshot isolation: every read uses the transaction's snapshot

Write skew: read shared data, write different rows, break a rule

SSI: Postgres serializable: detects dangerous read/write patterns

Serialization failure: abort to keep a serial order; retry it

7 Interview questions

Q1. What anomaly does repeatable read allow? BASIC

In the SQL standard, phantoms: a repeated query can return new rows. In snapshot-based implementations like Postgres and MySQL, phantoms are hidden from plain reads, but write skew is still possible.

Q2. What is write skew? BASIC

Two concurrent transactions read the same data, decide based on it, and update different rows. Neither conflicts with the other, but together they break an invariant, like both on-call doctors going off call.

Q3. What is the default isolation level in Postgres and MySQL? BASIC

Postgres (and Oracle, SQL Server): read committed. MySQL InnoDB: repeatable read, implemented with snapshots.

Q4. A meeting-room app lets two people book the same room for the same hour. Why, and what's the fix? SCENARIO

Each transaction checks “no overlapping booking”, finds none, and inserts its own row: write skew via a phantom. FOR UPDATE can't lock rows that don't exist yet. Use serializable, or an exclusion/unique constraint on (room, time slot) so the database rejects the second insert.

Q5. You switch a service to serializable. What must change in the code? SCENARIO

Every transaction needs a retry loop on serialization failure (SQLSTATE 40001), with a small limit such as 3–5 attempts and backoff. Keep transactions short to reduce aborts, and make the transaction body safe to re-run.

Q6. A finance report runs 5 queries and the totals don't add up. Why? SCENARIO

At read committed each statement sees a new snapshot, so commits between queries change the numbers. Run the report at repeatable read so all 5 queries share one snapshot.

Q7. Does snapshot isolation prevent lost updates? TRICKY

In Postgres, yes: if two repeatable-read transactions update the same row, the second gets a serialization error. At read committed, a read-modify-write in application code can still lose an update.

Q8. If serializable prevents everything, why isn't it the default? TRICKY

It costs throughput: tracking dependencies or holding locks adds overhead, and conflicting transactions are aborted and must retry. Under contention, abort rates rise. Most workloads are fine with read committed plus targeted locks.

Next: SD-E-010 · MVCC Explained backendroadmap.com